Laporan: Perbandingan IaaS, PaaS, dan SaaS — Studi Kasus JogjaTrip

Oct 26, 2026 · @Frans

Pendahuluan

Laporan ini mendemonstrasikan tiga model layanan cloud computing — Infrastructure as a Service (IaaS), Platform as a Service (PaaS), dan Software as a Service (SaaS) — dengan men-deploy satu aplikasi nyata, JogjaTrip, lewat dua cara berbeda (IaaS dan PaaS), sekaligus menunjukkan SaaS yang sudah terpakai di dalamnya.

JogjaTrip adalah aplikasi perencana perjalanan wisata Kota Yogyakarta berbasis Laravel 12 dan MySQL, dengan fitur katalog destinasi, itinerary builder multi-hari, rekomendasi dan itinerary berbasis AI, serta kolaborasi antar pengguna. Kompleksitas aplikasi ini membuat perbandingan ketiga model lebih realistis dibanding aplikasi contoh sederhana.

Tujuan laporan: membandingkan jumlah langkah, waktu setup, kontrol, biaya, dan masalah nyata yang ditemukan saat men-deploy JogjaTrip dengan IaaS dan PaaS, serta mengidentifikasi SaaS yang sudah dipakai di dalamnya.

Dasar Teori

Cloud computing, menurut definisi formal **NIST SP 800-145** (Mell & Grance, 2011), adalah model yang memungkinkan akses jaringan sesuai permintaan ke sekumpulan sumber daya komputasi (jaringan, server, storage, aplikasi, layanan) yang dapat disediakan dan dilepas dengan cepat, dengan usaha manajemen atau interaksi penyedia layanan yang minimal. Definisi ini mensyaratkan lima karakteristik esensial, dan tiga model layanan utama dibedakan oleh satu hal: siapa yang mengurus setiap lapisan teknologi.

Lima karakteristik esensial tersebut:

- **On-demand self-service** — pengguna menyediakan resource komputasi sendiri tanpa interaksi manusia dari pihak provider
- **Broad network access** — resource dapat diakses lewat jaringan standar dari berbagai jenis perangkat
- **Resource pooling** — resource komputasi provider dikumpulkan untuk melayani banyak pengguna sekaligus (multi-tenant)
- **Rapid elasticity** — kapasitas dapat ditambah/dikurangi secara elastis, kadang otomatis
- **Measured service** — pemakaian resource dipantau, dikontrol, dan dilaporkan secara transparan, umumnya dengan model bayar sesuai pakai

Kelima karakteristik ini dipakai sebagai alat ukur jujur untuk menilai apakah demo IaaS yang dijalankan benar-benar IaaS murni atau sekadar simulasi (lihat bagian Metodologi).

Model	Definisi	Yang kita urus	Yang provider urus
IaaS	Sewa infrastruktur komputasi mentah (virtual machine)	Aplikasi, data, runtime, middleware, OS	Virtualisasi, server fisik, storage, jaringan
PaaS	Sewa platform untuk menjalankan kode	Aplikasi & data	Runtime, middleware, OS, virtualisasi, server, storage, jaringan
SaaS	Memakai aplikasi jadi lewat browser	Tidak ada	Semua lapisan

Analogi yang sering dipakai untuk menjelaskan ini ke orang awam adalah "Pizza as a Service": memasak sendiri (on-premises), kit pizza beku (IaaS), pesan delivery (PaaS), dan makan di restoran (SaaS). Semakin ke kanan, semakin sedikit yang perlu dikerjakan sendiri, tapi semakin sedikit juga kontrol atas hasilnya.

Tools yang Dipakai

Aplikasi yang didemokan: JogjaTrip (Laravel 12, PHP 8.2, MySQL 8, Tailwind + Alpine.js lewat Vite). Kode sumber: github.com/anugrahhhhh/tripplane-jogja-itenary.

Kebutuhan	Tool / Provider	Peran dalam demo
VM (IaaS)	VirtualBox (VM lokal Ubuntu)	Dipakai setelah sign-up Azure for Students & Oracle Cloud terblokir kendala administratif (lihat bagian Hasil)
Web server & runtime (IaaS)	Nginx, PHP-FPM, MySQL 8	Diinstall & dikonfigurasi manual di VM
Platform (PaaS)	Railway	Build dari Dockerfile, database MySQL terkelola
Kontainerisasi	Docker (multi-stage build)	Dasar image untuk deploy PaaS
AI (SaaS)	Google Gemini API	Rekomendasi destinasi & generator itinerary otomatis

Kebutuhan	Tool / Provider	Peran dalam demo
Autentikasi (SaaS)	Google OAuth 2.0	Login dengan akun Google
Cuaca (SaaS)	Open-Meteo	Prakiraan cuaca per hari, gratis tanpa API key
Peta (SaaS)	OpenStreetMap + Leaflet.js	Peta lokasi destinasi & itinerary

Metodologi

Demo pada laporan ini disusun dengan pendekatan verifikasi langsung: setiap klaim keberhasilan dipastikan lewat observasi nyata (output terminal, kode status HTTP, tangkapan layar browser), bukan asumsi. Langkah sistematis yang ditempuh:

1. **Deploy nyata, bukan simulasi tertulis** — setiap tahap instalasi (Nginx, PHP, MySQL, dsb.) dijalankan sungguhan di VM, dengan output perintah dicatat apa adanya, termasuk error yang muncul.
2. **Verifikasi fungsional** — setelah deploy, aplikasi diakses lewat browser dan beberapa halaman kunci (katalog, detail destinasi, login) dipastikan merespons `HTTP 200` sebelum dinyatakan berhasil.
3. **Pencatatan kendala apa adanya** — setiap hambatan (termasuk kegagalan sign-up Azure for Students dan Oracle Cloud Always Free) dicatat dan dilaporkan, bukan disembunyikan, demi kejujuran akademik.
4. **Evaluasi terhadap standar formal** — hasil deploy diperiksa ulang terhadap definisi NIST SP 800-145 untuk menilai sejauh mana demo ini benar-benar memenuhi kriteria IaaS (lihat bagian Dasar Teori).

Langkah-Langkah Demo

Panduan lengkap dengan perintah siap pakai ada di repo: `docs/deployment/IAAS-GUIDE.md` dan `docs/deployment/PAAS-GUIDE.md`. Ringkasannya:

> **Catatan jujur soal definisi IaaS**: VM di bawah ini berjalan lokal lewat VirtualBox, bukan di cloud publik. Secara ketat ini bukan IaaS murni (tidak ada *broad network access* dari provider, tidak disewa/dibayar), tapi **proses teknisnya identik**: kita yang install OS, web server, runtime, dan database dari nol tanpa bantuan otomatis provider. Alasan dipakai VM lokal: sign-up Azure for Students terhambat verifikasi MFA akun kampus, dan Oracle Cloud Always Free terhambat rate-limit saat pendaftaran — keduanya kendala administratif, bukan kendala teknis.

IaaS (VM lokal VirtualBox, 7 tahap, dikerjakan manual lewat SSH):

1. Buat VM Ubuntu di VirtualBox (unattended install), aktifkan SSH server di dalamnya
2. Setup port forwarding VirtualBox (SSH 2222→22, HTTP 8080→80) supaya bisa diakses dari laptop
3. SSH masuk, install Nginx, PHP, MySQL, Composer, Node.js dari repo Ubuntu
4. Clone repo, `composer install`, `npm run build`
5. Isi `.env` (`APP_URL=http://localhost:8080`), `migrate --seed`, `storage:link`
6. Tulis konfigurasi Nginx sebagai reverse proxy ke PHP-FPM
7. Pasang cron untuk `schedule:run`, lalu verifikasi lewat `http://localhost:8080`

PaaS (Railway, 4 langkah, lewat dashboard web):

1. New Project → Deploy from GitHub repo (Railway membaca `railway.json`, build dari `docker/paas/Dockerfile`)
2. Add service → Database → MySQL
3. Isi environment variable (API key, referensi variabel database)
4. Railway build & deploy otomatis — aplikasi online dengan domain publik

[Tempel screenshot demo di sini: terminal SSH untuk IaaS, dan dashboard Railway untuk PaaS.]

Hasil dan Perbandingan

Kedua jalur deploy berhasil menjalankan JogjaTrip (katalog destinasi, login, rekomendasi AI, cuaca) dengan hasil akhir yang identik bagi pengguna, tapi proses dan trade-off di baliknya sangat berbeda.

Aspek	IaaS (VM lokal)	PaaS (Railway)
Jumlah langkah utama	7 tahap, puluhan command manual	4 langkah lewat dashboard
Waktu setup	[isi waktu aktual demo]	[isi waktu aktual demo]
Siapa urus OS & server	Kita, manual lewat SSH	Provider, otomatis
Kontrol	Tinggi — bebas ubah apa saja	Rendah — ikut batasan platform
Biaya (untuk demo)	Gratis (VM lokal, hardware sendiri)	Free tier Railway

Aspek	IaaS (VM lokal)	PaaS (Railway)
Masalah yang ditemukan	Sign-up Azure for Students terblokir verifikasi MFA kampus; Oracle Cloud Always Free terblokir rate-limit saat pendaftaran — akhirnya beralih ke VM lokal; PPA `ondrej/php` juga tidak kompatibel dengan versi Ubuntu yang dipakai	Foto destinasi hilang saat redeploy karena filesystem container bersifat ephemeral; scheduler harus memakai proses terpisah, bukan crontab sendiri

Temuan IaaS yang paling penting: rintangan terbesar ternyata bukan teknis, melainkan administratif. Sign-up Azure for Students gagal di tahap verifikasi MFA (Multi-Factor Authentication) akun kampus yang tidak bisa diselesaikan tanpa akses langsung ke perangkat terdaftar. Sign-up Oracle Cloud Always Free gagal karena rate-limit ("Forbidden: number of requests exceeded") yang persisten dari sistem anti-fraud mereka. Setelah dua jalur cloud publik gagal, demo dialihkan ke VM lokal dengan VirtualBox, yang berhasil dijalankan penuh. Ini menunjukkan bahwa "gratis" di cloud sering tetap punya gesekan nyata: verifikasi identitas, kebijakan institusi, dan rate-limiting anti-fraud — bukan cuma soal kemampuan teknis.

Temuan PaaS yang paling penting untuk didiskusikan: foto yang diupload admin disimpan di `storage/app/public` di dalam container. Setiap redeploy membuang container lama, sehingga foto yang diupload sebelumnya hilang. Solusi produksi yang sebenarnya adalah memindahkan storage ke layanan object storage (misalnya S3-compatible), di luar scope demo ini.

Data Kuantitatif dari Proses Deploy

Angka-angka berikut diambil langsung dari log terminal saat demo IaaS dijalankan, bukan perkiraan:

Metrik	Nilai
Paket npm ter-install (`npm ci`)	150 paket (151 diaudit)
Migrasi database dijalankan	14 migrasi
Destinasi ter-seed otomatis	72 destinasi
Waktu build Vite	2.12 detik (59 modules)
Kerentanan ditemukan (`npm audit`)	6 (1 moderate, 3 high, 2 critical)
Versi PHP-FPM terpasang	8.5

Metrik	Nilai
Versi Ubuntu	26.04.1 LTS
Versi Node.js	v22.22.1

Temuan 6 kerentanan dari `npm audit` layak didiskusikan: ini pengingat nyata bahwa dependency management dan patching berkala tetap penting terlepas dari model cloud yang dipakai — tanggung jawab ini ada di tangan developer (kita), bukan provider cloud mana pun, karena kerentanan ini ada di level kode aplikasi, bukan infrastruktur.

Analisis Keamanan

Model tanggung jawab yang sama untuk infrastruktur juga berlaku untuk keamanan — permukaan serangan (attack surface) berbeda tergantung siapa yang bertanggung jawab atas tiap lapisan.

Aspek Keamanan	IaaS (VM Lokal)	PaaS (Railway)
Patch OS & kernel	Kita, manual (`apt upgrade`)	Provider, otomatis
Patch runtime (PHP-FPM, Node.js)	Kita, manual	Provider, lewat rebuild image
Exposure ke internet publik	Tidak ada (hanya localhost)	Publik, langsung bisa diakses
Isolasi antar aplikasi/tenant	Tidak relevan, VM milik sendiri	Provider jamin isolasi container
Backup & disaster recovery	Tanggung jawab kita penuh	Tergantung provider, tidak otomatis
Respon terhadap kerentanan (CVE) baru	Kita pantau & patch sendiri	Provider, untuk layer di bawah kode

Poin penting: karena VM lokal ini hanya bisa diakses lewat `localhost` (tidak ada port yang terbuka ke internet), permukaan serangannya praktis nol dibanding IaaS cloud sungguhan yang memiliki IP publik. Ini salah satu trade-off tersembunyi dari memakai VM lokal sebagai simulasi — keamanannya tidak representatif dibanding VM cloud asli yang harus benar-benar dikeraskan (hardened) karena terekspos ke internet.

Kesimpulan

Tidak ada satu model yang selalu lebih baik; pilihannya tergantung seberapa besar tim, seberapa unik kebutuhan infrastruktur, dan seberapa penting kontrol penuh dibanding kecepatan deploy.

- **IaaS** cocok dipakai saat butuh kontrol penuh atas konfigurasi server, atau untuk belajar cara kerja infrastruktur dari dasar. Trade-off-nya: setup paling lama dan paling rawan salah konfigurasi.
- **PaaS** cocok dipakai saat tim ingin fokus ke kode, bukan infrastruktur. Trade-off-nya: kontrol terbatas dan storage container bersifat sementara.
- **SaaS** cocok dipakai untuk kebutuhan umum yang sudah tersedia sebagai layanan jadi (AI, autentikasi, cuaca, peta), sehingga tidak perlu dibangun ulang dari nol.

Untuk JogjaTrip sendiri, kombinasi yang paling masuk akal adalah PaaS untuk kemudahan maintenance jangka panjang, ditambah object storage terpisah untuk foto agar tidak hilang saat redeploy. IaaS tetap berharga sebagai latihan memahami apa yang sebenarnya terjadi di balik kemudahan PaaS.

Referensi

- Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing (SP 800-145). National Institute of Standards and Technology.
- Armbrust, M., et al. (2010). A View of Cloud Computing. *Communications of the ACM*, 53(4), 50–58.
- Repo JogjaTrip — kode sumber, `docs/deployment/IAAS-GUIDE.md`, `docs/deployment/PAAS-GUIDE.md`
- Microsoft Learn — Shared responsibility in the cloud
- Railway — Documentation
- Azure for Students
- Oracle Cloud Always Free
- Google Cloud Free Tier
- DigitalOcean
- Open-Meteo API
- Google AI Studio — Gemini API